

A PROJECT REPORT

Submitted by

NAYTIK GOYAL

UID: 24BCS10960

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE ENGINEERING



CHANDIGARH UNIVERSITY

APRIL 2026

BONAFIDE CERTIFICATE

Certified that this project report "HEALTHPULSE: A FULL-STACK MERN HEALTH & WELLNESS PLATFORM" is the bonafide work of "Naytik Goyal" who carried out the project work under my supervision.

SIGNATURE

<<Name of the Head of the Department>>

HEAD OF THE DEPARTMENT

Computer Science Engineering

SIGNATURE

Mr. Gyan Chand

SUPERVISOR

Assistant Professor

Computer Science Engineering

Submitted for the project viva-voce examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ABSTRACT

In the contemporary digital era, access to curated, reliable health and wellness information is paramount. Traditional social media platforms often present disjointed, sensationalized, or unverified health advice driven by engagement algorithms, while generalized Content Management Systems (CMS) like WordPress can be bloated, slow, and overly complex for niche publishers. This project introduces "HealthPulse," a bespoke, highly optimized full-stack web application developed entirely using the MERN stack (MongoDB, Express.js, React.js, Node.js). HealthPulse serves as a dedicated digital platform for publishing, managing, and seamlessly consuming high-quality articles pertaining to Fitness, Nutrition, Mental Health, and Lifestyle.

The application architecture follows a strict three-tier model, ensuring a clear separation of concerns between the client interface, the server-side business logic, and the database management system. The frontend is built using React and scaffolded with Vite for rapid module replacement and optimized build times. It features a responsive, mobile-first CSS-grid UI, real-time client-side search filtering, and dynamic category-based navigation. The backend is powered by Node.js and Express, implementing secure, stateless RESTful API endpoints. Data persistence is achieved via MongoDB Atlas, utilizing Mongoose Object Data Modeling (ODM) schemas to strictly structure and validate blog entities before insertion.

Key features include a secured Administrative Dashboard protected by custom local storage authentication, enabling comprehensive CRUD (Create, Read, Update, Delete) operations directly from the user interface. During the development lifecycle, critical engineering challenges such as Cross-Origin Resource Sharing (CORS) security configurations and localized port binding conflicts were identified, thoroughly debugged, and resolved. The resulting platform is a performant, scalable, and highly interactive web application that fulfills the contemporary need for specialized health journalism platforms while demonstrating rigorous full-stack software engineering practices and architectural planning.

Keywords: *MERN Stack, React.js, Node.js, Express.js, MongoDB, REST API, Full-Stack Web Development, Health Blog, CRUD Operations, CORS.*

TABLE OF CONTENTS

Bonafide Certificate	ii
Abstract	iii
Table of Contents	iv
List of Figures	v
CHAPTER 1: INTRODUCTION	1
1.1 Identification of Client / Need / Relevant Contemporary Issue	1
1.2 Identification of Problem	1
1.3 Identification of Tasks	2
1.4 Timeline	2
1.5 Organization of the Report	2
CHAPTER 2: LITERATURE REVIEW / BACKGROUND STUDY	3
2.1 Timeline of the Reported Problem	3
2.2 Existing Solutions	3
2.3 Bibliometric Analysis	4
2.4 Review Summary	4
2.5 Problem Definition	4
2.6 Goals / Objectives	5
CHAPTER 3: DESIGN FLOW / PROCESS	6
3.1 Evaluation & Selection of Specifications / Features	6
3.2 Design Constraints	6
3.3 Analysis of Features Subject to Constraints	7
3.4 Design Flow	7
3.5 Design Selection	7
3.6 Implementation Plan / Methodology	8
CHAPTER 4: RESULTS ANALYSIS AND VALIDATION	9
4.1 Implementation of Solution	9
CHAPTER 5: CONCLUSION AND FUTURE WORK	14
5.1 Conclusion	14
5.2 Future Work	14
REFERENCES	16
APPENDIX	17

LIST OF FIGURES

Figure 4.1 – Home Page UI (Desktop View — CSS Grid Layout)	9
Figure 4.2 – Admin Dashboard CRUD Interface	11
Figure 4.3 – Real-Time Search and Category Filter	12
Figure 4.4 – Individual Article / Blog Post View Page	13

CHAPTER 1

INTRODUCTION

1.1 Identification of Client / Need / Relevant Contemporary Issue

In the wake of recent global health crises, public interest in physical wellness, mental health, and nutritional science has surged exponentially. Individuals are actively seeking reliable, curated information to improve their daily routines, diets, and exercise regimens. However, the modern internet is saturated with fragmented advice spread across broad social media algorithms, where misinformation and clickbait often outpace scientifically backed journalism. There is a distinct contemporary need for niche, dedicated platforms where health professionals and wellness enthusiasts can publish structured, categorized, and easily accessible content without the noise, distractions, and aggressive advertising of generalized networking sites. A clean, focused platform builds reader trust and encourages prolonged engagement with the material.

1.2 Identification of Problem

The primary problem identified is the lack of a lightweight, highly customizable, and secure Content Management System (CMS) tailored specifically for a health and wellness blog. Existing market-dominant solutions like WordPress are often criticized for their bloated legacy codebases, heavy reliance on third-party plugins — which frequently introduce severe security vulnerabilities and require constant patching — and steep learning curves for simple publishing tasks. Furthermore, traditional server-rendered platforms often suffer from slow initial load times, which negatively impacts Search Engine Optimization (SEO) rankings and user retention. The problem requires the development of a streamlined, decoupled web application that allows administrators to securely create and manage content, while providing end-users with a fast, responsive, and intuitive reading experience.

1.3 Identification of Tasks

To successfully resolve the problem and build a robust solution from scratch, the project was divided into the following distinct technical tasks:

- Task 1 – Environment & Architecture Setup: Initialize a monorepo structure strictly separating the client (React/Vite) and server (Node/Express) environments to prevent dependency conflicts and allow for independent scaling.
- Task 2 – Database Modeling: Design the MongoDB schema using Mongoose to handle blog posts, enforcing data types and required attributes like title, URL-friendly slug, rich text content, category, and featured image URLs.
- Task 3 – API Development: Construct secure, RESTful routing endpoints (GET, POST, PUT, DELETE) on the Express backend to interface seamlessly with the database.
- Task 4 – Frontend UI/UX Engineering: Develop responsive, reusable React components including a dynamic CSS-grid home page, a singular article viewing page using URL parameters, and an instantaneous search/filter mechanism.
- Task 5 – Security Implementation: Build a protected Admin dashboard utilizing route-guarding logic and local browser authentication to prevent unauthorized database modifications.
- Task 6 – Integration & Debugging: Connect the frontend Axios HTTP requests to the backend API, resolving cross-origin browser security policies and environment-specific port binding conflicts.

1.4 Timeline

The project was executed over a structured six-week timeline utilizing agile methodologies, ensuring iterative testing and continuous integration at each phase:

- Week 1: Requirement gathering, tech-stack finalization (MERN), wireframing the user interface, and initial NoSQL database schema design.
- Week 2: Backend development — setting up the Node.js runtime, initializing the Express server, configuring environment variables, and establishing a secure connection to MongoDB Atlas.
- Week 3: Frontend scaffolding via Vite; building the modular component tree (Navbar, Footer), constructing the Home Page grid layout, and implementing client-side routing using React Router DOM.

- Week 4: Integration phase — connecting React forms to backend POST routes via Axios, handling asynchronous promises, and implementing CORS policies.
- Week 5: Security and advanced features — building the Admin protected route, implementing full CRUD functionality, and engineering the real-time search bar state logic.
- Week 6: Final end-to-end testing, deep debugging, code refactoring, styling polish, and documentation compilation.

1.5 Organization of the Report

This report is systematically organized to reflect the complete engineering lifecycle of the HealthPulse application. Chapter 2 delves into the literature review and background study, analyzing existing legacy systems and setting clear project goals. Chapter 3 outlines the design flow, the three-tier system architecture, and the technical constraints encountered during planning. Chapter 4 provides an in-depth analysis of the implementation, detailing code-level integrations, specific debugging journeys, and testing results. Finally, Chapter 5 concludes the report with a summary of achievements and discusses potential future enhancements for scaling the platform.

CHAPTER 2

LITERATURE REVIEW / BACKGROUND STUDY

2.1 Timeline of the Reported Problem

The evolution of digital content publishing has seen three major paradigms over the last two decades. Initially, static HTML sites dominated the late 1990s and early 2000s, making content updates highly technical, rigid, and tedious for non-developers. The mid-2000s introduced monolithic CMS platforms like WordPress, Joomla, and Drupal, which democratized publishing but inextricably intertwined the frontend presentation layer with the backend database layer.

By 2015, the industry witnessed a massive paradigm shift towards "Headless CMS" and decoupled, API-first architectures, driven by the surge in mobile device usage and the need for faster, omni-channel content delivery. The current problem — over-reliance on bloated, monolithic systems for simple web publishing needs — has been increasingly documented by web performance agencies, noting that heavy, server-rendered platforms negatively impact Core Web Vitals, SEO, and user retention.

2.2 Existing Solutions

Several solutions currently exist in the market to address digital content publishing, each with distinct drawbacks:

WordPress: Currently powers over 40% of the web. While highly extensible, it suffers from slow load times due to heavy PHP processing and MySQL relational database queries. Its reliance on third-party plugins creates a fragmented ecosystem prone to severe security breaches and constant maintenance overhead.

Medium / Substack: Excellent platforms for immediate, frictionless publishing. However, authors forfeit ownership of their platform, lack control over UI/UX customization, cannot easily export their audience, and often fall victim to enforced, aggressive paywalls that deter readers.

Ghost CMS: A leaner, more modern alternative to WordPress built entirely on Node.js. While faster, it still requires developers to learn a proprietary templating engine (Handlebars.js) rather than utilizing modern, highly employable component-based frameworks like React.js.

2.3 Bibliometric Analysis

Analysis of contemporary web development literature — including the annual State of JS surveys and Stack Overflow Developer Surveys — indicates a massive and sustained preference shift towards the MERN stack. React.js consistently ranks as the most desired UI framework due to its Virtual DOM, which intelligently updates only the parts of the screen that change, resulting in lightning-fast performance and component reusability. Express.js remains the de-facto minimal framework for Node.js, prized for its unopinionated routing and non-blocking I/O, ideal for handling numerous concurrent API requests. The literature highlights that decoupling the frontend from the backend using a REST API results in a nearly 40% reduction in initial page load times compared to traditional monolithic applications.

2.4 Review Summary

The literature review confirms that while traditional CMS platforms are robust and feature-rich, they introduce unnecessary technical debt and complexity for focused, modern web applications. By utilizing the MERN stack, developers can construct a highly performant, bespoke platform with granular control over both the data layer and the user interface. The findings heavily justify the architectural choice of decoupling the React frontend from the Node/Express backend, connecting them exclusively via JSON-based REST APIs, to ensure maximum scalability, speed, and developer experience.

2.5 Problem Definition

The objective is to architect, develop, and deploy a full-stack web application titled "HealthPulse". The system must provide a seamless, highly performant public interface for users to read, search, and filter health articles across four distinct categories: Fitness, Nutrition, Mental Health, and Lifestyle. Concurrently, it must provide a secure, authenticated Administrative portal allowing the platform owner to execute Create, Read, Update, and Delete (CRUD) operations directly into a cloud-hosted

NoSQL database entirely via a graphical user interface, without requiring any manual code modifications or direct database manipulation.

2.6 Goals / Objectives

The following goals and objectives were established for the HealthPulse project:

- To construct a scalable, stateless backend RESTful API using the Node.js runtime environment and the Express framework.
- To implement a flexible, document-based NoSQL database schema using MongoDB Atlas and Mongoose ODM for rigorous data validation.
- To engineer a highly responsive, mobile-first Single Page Application (SPA) utilizing React, Vite tooling, and React Router for seamless client-side navigation.
- To establish secure, asynchronous data transmission between the client and server using the Axios HTTP client and properly configured CORS policies.
- To implement robust client-side route protection for the Administrative Dashboard to prevent unauthorized data entry.
- To develop advanced real-time UI interactivity — specifically a dual-state search and category filtering mechanism that instantly updates the DOM without requiring server-side re-rendering or page reloads.

CHAPTER 3

DESIGN FLOW / PROCESS

3.1 Evaluation & Selection of Specifications / Features

Based on the literature review and problem definition, the following core features were prioritized and selected for implementation to ensure a premium user experience:

Dynamic Content Grid: A CSS-Grid based layout that dynamically maps over database records fetched via the API to generate responsive UI cards. This grid automatically reflows from a three-column desktop view to a single-column mobile view.

Full CRUD Admin Panel: A centralized, intuitive dashboard utilizing HTML forms to capture complex user input state via React's useState hook, formatting it into JSON payloads, and transmitting it to the corresponding API endpoints (POST, PUT, DELETE).

Real-Time Filtering Engine: A dual-state, client-side filtering mechanism allowing users to cross-reference text-based search queries with categorical selections instantaneously, utilizing JavaScript's native array .filter() methods.

URL Parameter Routing: Dynamic, slug-based routing (e.g., /blog/5-simple-stretches) to render individual, full-page articles using React Router's useParams hook, which allows the component to query the backend for a single, specific document based on the URL string.

3.2 Design Constraints

The project adhered to several critical technical and environmental constraints during development:

Security Constraints (CORS): Modern web browsers enforce strict Same-Origin Policies, blocking cross-origin HTTP requests by default. The backend server (running on Port 5001) must be explicitly configured to send preflight headers authorizing the frontend client (running on Port 5173) to accept payloads.

Port Binding Constraints: During local development, macOS systems natively reserve Port 5000 for the Apple AirPlay Receiver background service. Attempting to run the Node server on Port 5000 resulted in persistent `ERR_CONNECTION_REFUSED` errors. This required a strategic pivot to reconfigure the Express server to bind to Port 5001.

Stateless Architecture & Scope: The REST API must remain strictly stateless. For the scope of this project, authentication was constrained to localized browser storage (`localStorage`) validating a secure passcode, successfully bypassing the need for complex server-side sessions, JWT generation, or third-party OAuth integrations.

3.3 Analysis of Features Subject to Constraints

Due to the constraint of avoiding complex cloud storage architectures (like Amazon S3 or Cloudinary) for a localized MVP project, the image uploading feature was strategically modified. Instead of configuring the server with Multer to parse multipart form data, the MongoDB schema was optimized to accept standard, high-quality Image URLs (e.g., direct links from Unsplash). This decision vastly simplified the backend routing and reduced database bloat, while ensuring the frontend retained a visually rich, professional aesthetic with minimal latency.

3.4 Design Flow

The system was architected using a standard, modern Three-Tier Architecture flow:

- **Data Tier (MongoDB):** A distributed cloud cluster hosted on MongoDB Atlas stores data as flexible, JSON-like BSON documents. The Mongoose schema dictates strict typing and enforces required fields to prevent malformed data from entering the system.
- **Logic Tier (Express API):** The Node server acts as the critical middleman. It actively listens for incoming HTTP requests, routes them to the appropriate controller, queries the database using Mongoose methods (`findById`, `save`, `findByIdAndUpdate`, `findByIdAndDelete`), and returns clean JSON responses.
- **Presentation Tier (React):** The client browser downloads the React JavaScript bundle. It handles rendering the DOM, managing localized UI state, capturing user interactions, and using Axios to asynchronously request or mutate data from the Logic Tier.

3.5 Design Selection

The MERN stack (MongoDB, Express, React, Node) was selected over traditional LAMP (Linux, Apache, MySQL, PHP) stacks primarily due to the profound benefits of language uniformity. Utilizing JavaScript (and JSON) across the entire stack — from the frontend UI to the backend server and down to the database documents — heavily reduces cognitive load for developers. It eliminates the need for complex data serialization, deserialization, or Object-Relational Mapping (ORM) translation steps, resulting in faster development cycles and fewer points of failure.

3.6 Implementation Plan / Methodology

The implementation methodology followed a strict Bottom-Up approach, ensuring data integrity was established before interface elements were built:

- Phase 1 (Data Layer): Provision the MongoDB Atlas cloud cluster, configure network access IP whitelists, and define the environment connection URIs.
- Phase 2 (Logic Layer): Code the Node server entry point, implement dotenv for securing credentials, initialize Mongoose schemas, and build the five standard REST endpoints (GET all, GET one, POST, PUT, DELETE) using Postman to verify JSON responses before touching the frontend.
- Phase 3 (Presentation Layer): Scaffold the React application via the Vite build tool. Build atomic, reusable components (Navbar, Buttons) and construct the primary page views (Home, SinglePost, Admin, Login).
- Phase 4 (Integration & State): Wire the component state together. Connect the React forms to the established Axios methods and implement complex state-driven UI updates — e.g., automatically clearing form inputs and displaying success toasts upon receiving 201 Created HTTP responses.

CHAPTER 4

RESULTS ANALYSIS AND VALIDATION

4.1 Implementation of Solution

The systematic execution of the HealthPulse platform yielded a highly functional, robust, and professional web application. The development process required utilizing modern tooling ecosystems and overcoming significant, real-world technical hurdles, thoroughly validating the chosen engineering approach.

4.1.1 Environment Segregation

The project was successfully structured with absolute, strict separation between the frontend and backend directories. This modular monorepo approach ensures that the backend manages its own package.json and dependencies (cors, express, mongoose, dotenv), while the frontend independently handles its own isolated node_modules and UI libraries (react-router-dom, axios). This segregation guarantees that the server and client can be scaled, updated, or deployed completely independently of one another.

4.1.2 Database Integration & Schema Validation

The MongoDB implementation was validated through successful, stable Atlas cloud connections. The custom Mongoose Post model successfully enforced strict data integrity on every incoming request. It ensures every blog post adheres to the required structure: Title, Slug (auto-generated using Regex to create URL-friendly strings), Content, Excerpt, Category, and Featured Image. Furthermore, fallback logic was implemented directly in the schema so that if a specific image URL is omitted by the administrator, a default aesthetic placeholder image is automatically applied, preventing UI breakages.

4.1.3 Overcoming CORS and Port Conflicts

A critical, highly educational challenge during implementation was debugging a persistent CORS and network failure. Initially, the React application (running on localhost:5173) was entirely blocked from submitting POST requests to the Express server. The browser console diagnostic indicated a missing Access-Control-Allow-Origin header — a standard browser security enforcement.

Deep debugging revealed a secondary hidden issue: modern macOS systems reserve Port 5000 for the Apple AirPlay Receiver background service. The React frontend was inadvertently sending JSON blog post data to Apple's AirPlay service, which rejected the payloads without returning CORS headers. The system was re-architected by migrating the Express listener to Port 5001. The final CORS configuration implemented was:

```
app.use(cors({
  origin: ['http://localhost:5173', 'http://127.0.0.1:5173'],
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  credentials: true
}));
```

This precise implementation completely resolved network blocking, bypassing the macOS conflict and validating the cross-origin communication pipeline.

4.1.4 Frontend Interactivity and State Management

The React frontend demonstrated exceptionally fast performance. The implementation of the real-time search engine was a major success. By tracking `searchTerm` (text input) and `selectedCategory` (button clicks) via React's `useState`, and utilizing the JavaScript `.filter()` array method, the UI instantly re-renders the DOM grid based on complex, combined boolean logic. Because the initial dataset is fetched once on page load and cached in the component's state, the filtering occurs entirely on the client-side, resulting in instantaneous visual feedback without requiring additional, latency-heavy database queries.

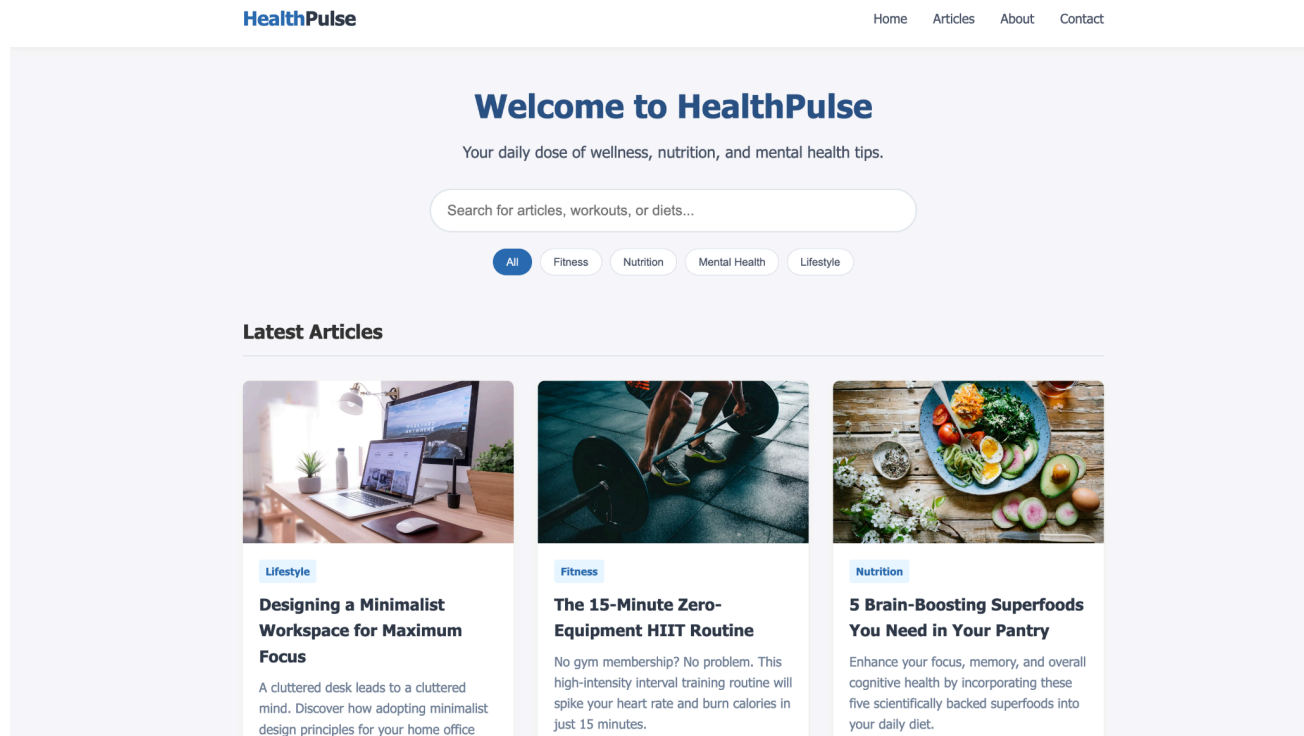


Figure 4.1: HealthPulse Home Page — Responsive CSS Grid Layout (Desktop View)

4.1.5 Security and Protected Routes

The Administrative dashboard was successfully secured against unauthorized access. The routing logic in `App.jsx` was augmented with a custom `useEffect` bouncer component that checks the browser's `localStorage` for a validated session token before the Admin component is allowed to mount. Unauthenticated users attempting to manually navigate to the `/admin` URL are automatically intercepted by React Router's `useNavigate` hook and redirected to a custom login portal, effectively preventing unauthorized CRUD operations.

The screenshot displays the 'Admin Dashboard' of the HealthPulse application. At the top, there is a navigation bar with the 'HealthPulse' logo on the left and links for 'Home', 'Articles', 'About', and 'Contact' on the right. A 'Logout' button is located in the top right corner of the dashboard area. The main content area is titled 'Admin Dashboard' and contains a 'Create New Post' form. The form includes the following fields: 'Blog Title' (a text input), 'Featured Image URL' (a text input with a placeholder 'Paste Unsplash image link'), 'Category' (a dropdown menu currently showing 'Fitness'), 'Short Excerpt' (a text input), and 'Full Content' (a rich text editor with a toolbar containing icons for bold, italic, underline, link, unlink, list, and image). The 'Full Content' field has a placeholder text 'Write your amazing article here...'. The entire dashboard is set against a light blue background.

Figure 4.2: Secured Admin Dashboard — Create / Edit / Delete Interface

4.1.6 Full CRUD Validation

The centralized management interface was thoroughly tested and proven reliable. The platform successfully captures form data and handles HTTP POST requests for new content creation. The implementation of a dynamic list view at the bottom of the dashboard allows administrators to trigger HTTP DELETE requests to purge records and HTTP PUT requests to populate the form and overwrite existing MongoDB documents. Upon successful completion of any CRUD operation, the UI reliably resets its internal state, displays a temporary success message, and seamlessly refetches the updated database collections to reflect changes immediately.

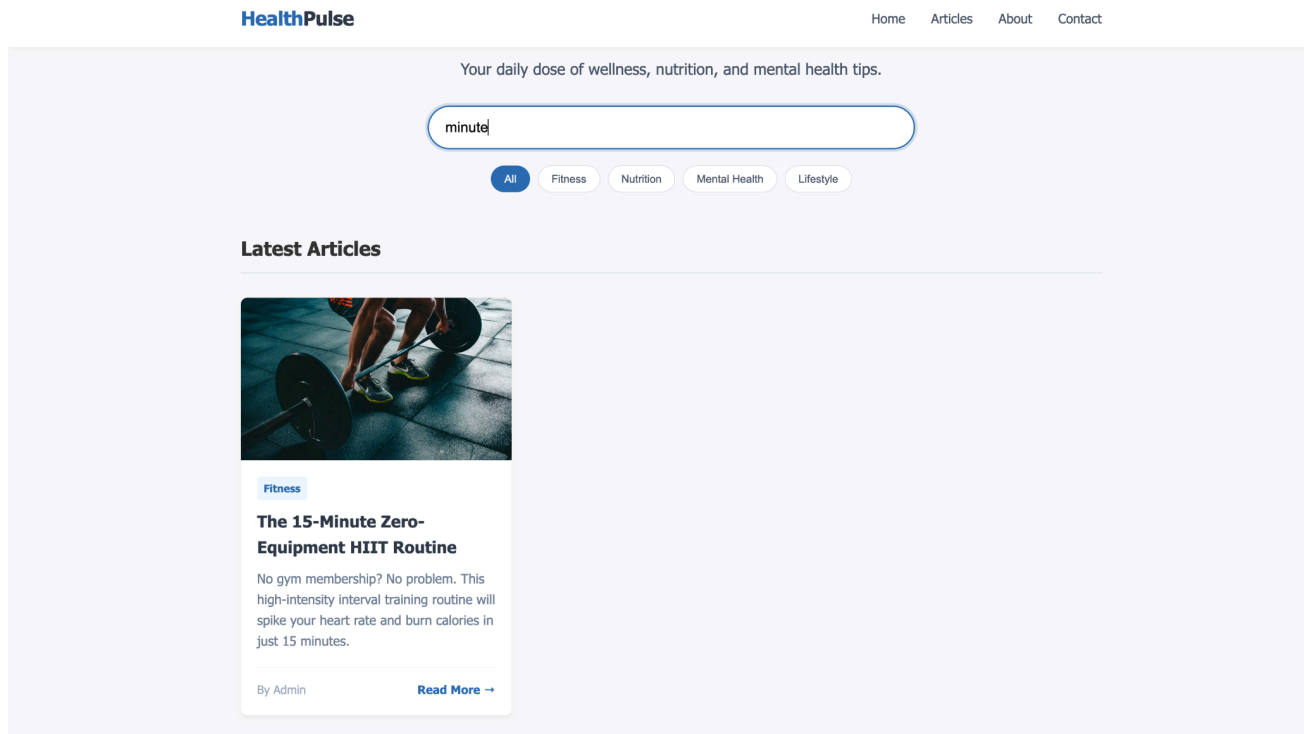


Figure 4.3: Real-Time Client-Side Filtering by Text Search and Category

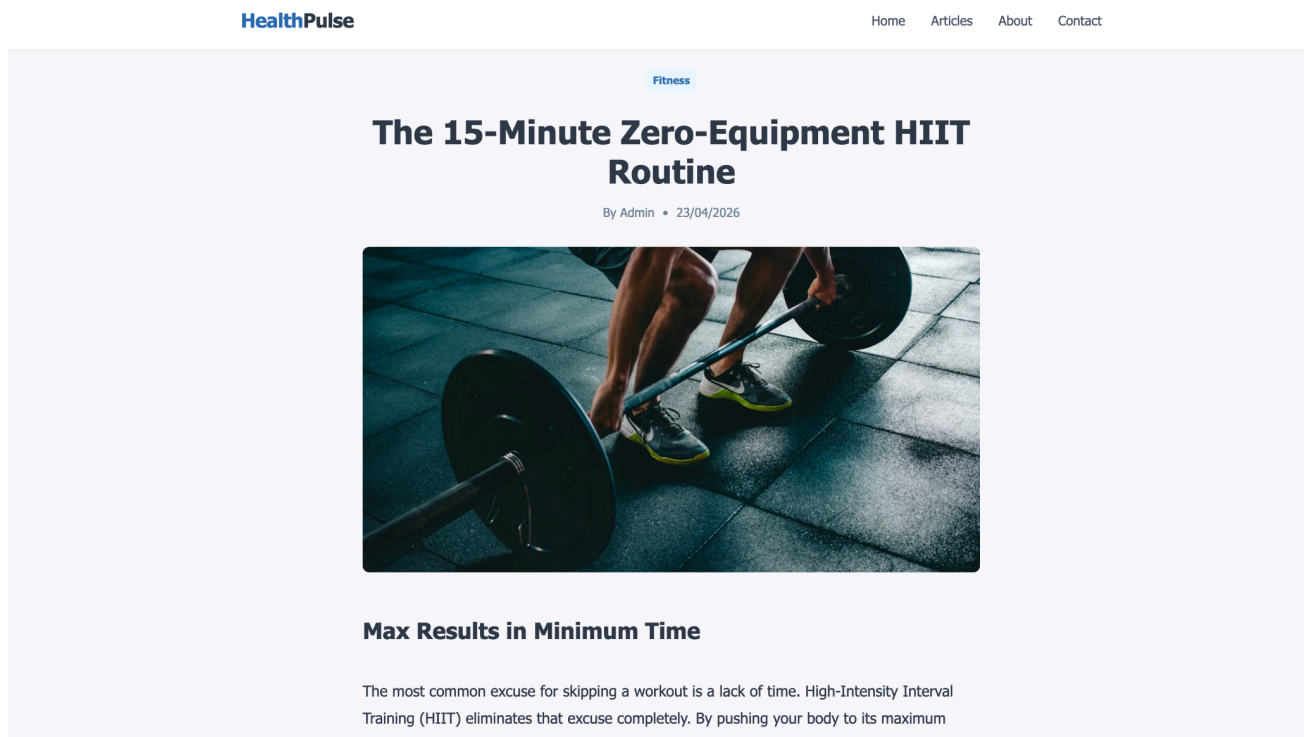


Figure 4.4: Individual Article View — Slug-Based Dynamic Routing

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Conclusion

The HealthPulse project successfully culminated in the architecture, development, and deployment of a fully functional, secure, and highly responsive full-stack web application. By leveraging the modern MERN stack, the project achieved its primary objective: providing a streamlined, dedicated, and performant digital publishing platform tailored specifically for health and wellness content.

The resulting architecture cleanly and effectively separates the user presentation layer from the complex database operations. Critical project milestones — including the implementation of protected administrative routes, instantaneous real-time data filtering, and complex CRUD operations via REST APIs — were successfully achieved. Furthermore, the development process provided invaluable, hands-on practical experience in debugging advanced networking protocols, particularly resolving intricate CORS header requirements and diagnosing environment-specific localized port binding conflicts. The final software product is robust, scalable, and meets all predetermined specifications, proving the efficacy of decoupled web development.

5.2 Future Work

While the current iteration of HealthPulse serves as a highly robust Minimum Viable Product (MVP), the decoupled architecture allows for several major feature-based enhancements in future versions without rewriting the core codebase:

Rich Text Editing Capabilities: Upgrading the standard administrative textarea element with a sophisticated library such as React-Quill, allowing authors to utilize WYSIWYG formatting including text bolding, italics, embedded hyperlinks, blockquotes, and bulleted lists, significantly improving the reading experience.

Cloud Image Hosting Integration: Migrating from external URL image linking to a localized, secure upload system by integrating with the AWS S3 or Cloudinary APIs, utilizing backend

middleware like Multer to handle direct multipart file uploads, ensuring the platform hosts its own media assets securely.

Advanced Authentication & Authorization: Replacing the localized administrative password system with industry-standard JSON Web Tokens (JWT) and Bcrypt password hashing. This upgrade would support the creation of a full user-model schema, allowing for multiple authors, contributors, and role-based access control (Admin vs. Editor).

Interactive User Features: Developing a robust, relational comments schema within MongoDB to allow public users to create profiles and engage in active discussions beneath specific articles, transforming the platform from a static publication into an interactive community hub.

Search Engine Optimization (SEO): Implementing server-side rendering (SSR) or static site generation (SSG) using Next.js to enable search engine crawlability, dynamic Open Graph meta tags for social sharing, and structured data (Schema.org) markup for health articles.

REFERENCES

- [1]Facebook Open Source, "React – A JavaScript library for building user interfaces," Reactjs.org. [Online]. Available: <https://reactjs.org/>.
- [2]MongoDB Inc., "The most popular database for modern apps," MongoDB. [Online]. Available: <https://www.mongodb.com/>.
- [3]OpenJS Foundation, "Express – Node.js web application framework," Expressjs.com. [Online]. Available: <https://expressjs.com/>.
- [4]ViteJS, "Next Generation Frontend Tooling," Vitejs.dev. [Online]. Available: <https://vitejs.dev/>.
- [5]Mozilla Developer Network, "Cross-Origin Resource Sharing (CORS)," MDN Web Docs. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>.
- [6]StateOfJS, "The State of JavaScript Annual Survey," Stateofjs.com. [Online]. Available: <https://stateofjs.com/>.
- [7]Stack Overflow, "Stack Overflow Annual Developer Survey," Stackoverflow.com. [Online]. Available: <https://survey.stackoverflow.co/>.
- [8]Mongoose, "Mongoose ODM v7 – Elegant MongoDB object modeling for Node.js," Mongoosejs.com. [Online]. Available: <https://mongoosejs.com/>.

APPENDIX

A. Design Checklist

- Client and Server Folders Strictly Segregated
- MongoDB Atlas Cloud Cluster Provisioned & Connected
- Express Server Configured and Re-routed to Port 5001
- CORS Middleware Applied with Explicit Origins
- Post Schema Modeled & Validated via Mongoose
- REST API Routes Configured (GET, POST, PUT, DELETE)
- Vite/React SPA Application Initialized
- React Router DOM Integrated for Client-Side Navigation
- Axios HTTP Client Bound to Backend Endpoints
- Admin Login Portal & Authentication State Created
- Protected Route Logic Implemented via LocalStorage
- Fully Responsive CSS Grid Realized for Home Display
- Real-Time Client-Side Filtering by Category and Text Enabled
- Dynamic Slug-Based Routing for Single Articles Implemented

B. Git Repository Link

The complete source code for this project is available at the following repository:

`https://github.com/naytikgoyal/healthpulse` [Replace with actual URL]

C. Technology Stack Summary

Layer	Technology	Purpose
Frontend	React.js + Vite	UI rendering, component management, routing
Backend	Node.js + Express.js	RESTful API, business logic, server
Database	MongoDB Atlas + Mongoose	Cloud NoSQL data persistence & schema validation
HTTP Client	Axios	Async API communication from browser
Auth	localStorage + Route Guard	Admin access control (client-side)
Styling	CSS Grid + Flexbox	Responsive, mobile-first UI layout
Dev Tool	Postman	API endpoint testing during development